

ECE 759 - Assignment 2

GitHub: <https://github.com/Hihi142/repo759/tree/main/HW02>

Host: euler145.engr.wisc.edu | Slurm job: 64262

UTC start: 2026-09-19T01:41:35.668145+00:00

Compiler: g++ (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4)

Build flags: -Wall -O3 -std=c++17. No environment modules loaded. Timings cover only the algorithm calls.

1. Inclusive scan

Tested all 21 powers of two from 2^{10} through 2^{30} , with one run per size. The labelled scaling plot is supplied separately as **task1.pdf**; task1.csv contains the measured time and first/last output elements for every size.

2. Convolution

For $n = 256$ and $m = 3$: time = 1.384336 ms; first output = 1.26428223; last output = -0.686184525. The correctness checks include the assignment's example, an asymmetric mask, edge/corner padding, masks larger than the image, and random inputs.

3. Matrix multiplication

A and B are 1024×1024 matrices. The four implementations passed independent elementwise correctness checks on smaller matrices; the full-size runs also have matching last elements within floating-point tolerance.

Function	Loop order	Time (ms)	Last element of C
mmul1	i, j, k	13913.010489	2.6878021150461802
mmul2	i, k, j	461.622315	2.6878021150461766
mmul3	j, k, i	35111.511716	2.6878021150461766
mmul4	i, j, k (vector)	16443.945507	2.6878021150461802

Observed order, fastest to slowest: mmul2, mmul1, mmul4, mmul3. Relative to mmul1, the elapsed-time ratios for mmul2, mmul3, and mmul4 are 0.033, 2.524, and 1.182, respectively.

With row-major storage, mmul1 traverses a row of A and a strided column of B in its inner loop. In mmul2, the inner j loop walks contiguous rows of B and C while reusing $A[i,k]$, improving spatial locality and making vectorization easier. The inner i loop in mmul3 strides through both A and C by 8192 bytes, which can increase cache misses and memory traffic.

mmul4 uses the same loop order as mmul1, and `std::vector` also stores elements contiguously. Its vectors are passed by const reference, with conversion outside the timer, so similar performance is expected. A single-run timing difference does not isolate container overhead from compiler effects and system variability.

The raw outputs and metadata.json accompany these results. The maximum difference between the four reported last elements is $3.55e-15$, consistent with floating-point rounding.